Да, бро. Из твоего промпта можно собрать **реально сильную новую серию обновлений**, но часть пунктов уже устарела относительно текущего RC51, поэтому я бы не переносил всё буквально.

Главный следующий шаг я бы назвал:

SERIES E — PRODUCT INTELLIGENCE & STUDIO COMPLETION

Цель этой серии: превратить AZRAIL из технически мощной системы в продукт, где **пользователь всегда понимает, что делать дальше**, а студии ощущаются как законченные профессиональные инструменты.

Самый важный пункт — именно то, что ты сказал про Google AI Studio: **контекстные подсказки прямо в Chat/Studio**, которые меняются в зависимости от проекта, файла, текущей ошибки, выбранной модели и стадии работы.

E01. Contextual Prompt Suggestions — приоритет №1

Не обычные статические кнопки:

Напиши код
Создай изображение
Объясни

а интеллектуальная система:

текущий проект
+
активная студия
+
выбранные файлы
+
последний ответ агента
+
ошибки
+
история миссии
+
доступные инструменты

+
бюджет
+
права пользователя
↓
Suggestion Engine
↓
3–6 следующих действий

Например, пользователь находится в Web Studio после генерации страницы:

[Проверить мобильную версию]
[Найти проблемы доступности]
[Сделать дизайн компактнее]
[Запустить browser QA]
[Открыть код компонента]
[Подготовить deploy]

В Music Studio:

[Проанализировать структуру трека]
[Создать MIDI-план]
[Проверить громкость]
[Разделить на stems]
[Сравнить с reference]

В Chat после ошибки:

[Исправить ошибку]
[Объяснить причину]
[Проверить связанные файлы]
[Откатить последнее изменение]

Архитектура

Я бы ввёл:

```
ui/components/suggestions/
  ├─ PromptSuggestionBar
  ├─ SuggestionChip
  ├─ SuggestionMenu
  ├─ SuggestionSkeleton
  └─ SuggestionErrorState

src/services/suggestions/
  ├─ suggestion-engine.ts
  ├─ suggestion-ranking.ts
  ├─ suggestion-context.ts
  └─ suggestion-policy.ts
```

Контракт:

```
SuggestionContext {
  surface
  projectId
  studio
  selectedFiles
  selectedElement
  lastUserMessage
  lastAgentResult
  activeModel
  capabilities
  errors
  budgetState
}
```

Suggestion Engine сначала должен использовать **дешёвые deterministic rules**, а LLM подключать только когда действительно нужен интеллектуальный вариант.

Это убирает лишние AI-вызовы.

E02. Smart Empty States

Сейчас пустой экран не должен быть просто:

No project selected

Он должен объяснять следующий шаг.

Например:

Image Studio

Что хотите создать?

[Product image]
[UI mockup]
[Social creative]
[Edit existing image]
[Generate from reference]

Web Studio

Начните с:

[Новый сайт]

[Импорт проекта]
[Анализ существующего UI]
[Создать страницу]
[Исправить responsive]

Chat

Попробуйте:

[Проанализировать проект]
[Найти ошибки]
[Создать новую функцию]
[Продолжить последнюю миссию]

E03. Universal Studio Toolbar

Твой пункт здесь правильный.

Не нужно делать отдельный toolbar для каждой студии.

Нужен:

StudioToolbar

Его стандарт:

Back
Project
Save status
Undo
Redo
Mode
Model
Run
Stop
History
Export
Settings
More

Но каждая студия показывает только релевантные действия.

Например, Music Studio:

Play
Stop
Render

Export

Web Studio:

Edit
Preview
Inspect
QA
Deploy

Image Studio:

Generate
Edit
Variants
Upscale
Export

E04. Save / Sync State

Очень важная вещь, которой часто не хватает AI-продуктам.

Всегда показывать:

Saving…
Saved
Syncing…
Offline
Conflict
Failed to save

И никогда не оставлять пользователя в сомнении:

Сохранилось или нет?

E05. Undo / Redo Engine

Не просто UI-кнопки.

Нужен настоящий command stack:

Action
↓

Command
↓
Apply
↓
History
↓
Undo / Redo

Для:

UI edits
component edits
architecture graph
studio settings
prompt parameters
module layout

Не применять Undo к необратимым внешним действиям вроде:

payment
deploy
external API mutation

Для них отдельный approval/recovery flow.

E06. History / Timeline

Твой пункт тоже правильный.

Нужна правая боковая панель:

History

14:42 Changed model
14:41 Updated Header.tsx
14:38 Generated landing page
14:34 Created project

[Restore]
[Compare]
[Open diff]

Но History должна быть связана с существующей event/release архитектурой, а не быть ещё одним независимым state store.

E07. Model Quick Switcher

В каждой AI-студии:

GPT
Claude
Gemini
Llama
Qwen
Mistral
...

Но не делать просто красивый dropdown.

Для каждой модели показывать:

Capability
Context
Speed
Cost
Provider
Available / unavailable

Пример:

Qwen Coder

Best for: Code
Context: 128K
Latency: Fast
Cost: Free tier
Status: Available

И Suggestion Engine должен учитывать активную модель.

E08. Model Auto-Router

Следующий уровень:

AUTO

Вместо того чтобы пользователь каждый раз выбирал модель.

Пример:

coding
→ coding model

image reasoning
→ multimodal model

long architecture analysis
→ high-context model

При этом пользователь всегда может увидеть:

Selected automatically:
Qwen Coder

Reason:
Coding task + Free profile

E09. Universal Module Actions

Твой `SortableModule` сценарий хороший.

Стандарт каждого модуля:

Grip
Collapse
Expand
Duplicate
Pin
Settings
Reset
Delete

Но `Delete` должен появляться только если действие реально допустимо.

E10. Studio State Machine

Каждая студия должна иметь единый lifecycle:

IDLE
READY
RUNNING
PAUSED

CANCELLING
FAILED
COMPLETED

UI автоматически меняется.

Например:

RUNNING

[Pause]
[Stop]
View logs

а не показывает одновременно бессмысленные:

Run
Run again
Deploy
Reset

E11. Real AbortSignal

Это один из действительно важных пунктов твоего списка.

Сейчас следующая серия должна обеспечить:

User presses Stop
↓
AbortController
↓
Mission
↓
Provider adapter
↓
HTTP request
↓
actual cancellation

А не:

Stop requested
→ wait until model finishes anyway

Это влияет и на UX, и на расходы.

E12. uncertain_effect Reconciliation

Это тоже очень сильный пункт.

Если внешний API получил:

Deploy
Create
Send
Purchase
Upload

но AZRAIL не получил ответ:

UNKNOWN

нельзя автоматически повторять.

Ввести:

effect_state

NOT_SENT
SENT
CONFIRMED
FAILED
UNCERTAIN
RECONCILED

И reconciliation workers.

E13. Error Boundary + Recovery

Во всех крупных пространствах:

Chat
Music Studio
Video Studio
Image Studio
Web Studio
Architecture
Labs

нужен отдельный Error Boundary.

Не просто:

Something went wrong.

а:

Studio encountered an error

Your work is safe.

[Retry]
[Restore last stable state]
[Open logs]
[Report]

E14. Connection Health

Для провайдеров и интеграций:

Connected
Degraded
Rate limited
Authentication expired
Unavailable

И кнопка:

Test connection

Но тест должен идти через backend.

API key никогда не должен проверяться браузером напрямую.

E15. Connector Gateway

Здесь я бы изменил твою стратегию.

Не надо делать сразу 31 адаптер.

Это создаст гигантский технический долг.

Сначала:

Connector Gateway

универсальный интерфейс:

```
ConnectorAdapter {
  connect()
  health()
  listCapabilities()
  execute()
  cancel()
  reconcile()
}
```

И только потом первые стратегические интеграции:

Cloudflare
GitHub
Figma
Canva
Google Drive
Slack

После стабилизации gateway остальные подключения становятся намного дешевле.

E16. Connector UI

Единая страница:

Connections

Cloudflare Connected
GitHub Connected
Canva Available
Figma Not connected
Google Drive Not connected
Slack Not connected

Для каждого:

Connect
Disconnect
Test
Permissions
Last sync
Health
Logs

E17. Web Studio — Design Contract

Очень хороший пункт из твоего промпта.

До генерации интерфейса должен появляться структурный контракт:

```
{
  "typography": {},
  "spacing": {},
  "colors": {},
  "grid": {},
  "components": {},
  "breakpoints": {},
  "interactionStates": {},
  "emptyStates": {},
  "errorStates": {}
}
```

AI сначала создаёт контракт.

Потом код.

Не наоборот.

E18. Visual ↔ Code Editing

Это одна из функций, которая реально может сильно выделить AZRAIL.

Пользователь кликает карточку в preview:

Selected:
PricingCard

и пишет:

Сделай компактнее

Система должна определить:

component
file
line
props
design tokens

и изменить именно исходный компонент.

Не генерировать новый HTML поверх старого.

E19. Layers / Component Tree

Для:

Web
Creative
Architecture
Image

нужна боковая панель:

Page
├ Header 👁 🔒
├ Hero 👁
├ Features
│ ├ FeatureCard
│ ├ FeatureCard
│ └ FeatureCard
└ Footer

Действия:

show/hide
lock
rename
duplicate
select
reorder

E20. Responsive Editing

Desktop/mobile не должны быть одной панелью разного размера.

Режимы:

Desktop
Tablet
Mobile

И mobile editing должен иметь адаптированные controls.

E21. Browser QA Agent

После изменения сайта:

Build
↓
Browser QA

Проверки:

mobile overflow
broken layout
forms
keyboard navigation
contrast
focus states
empty states
error states
console errors
failed requests

Результат:

QA score 91/100

Critical 0
Major 2
Minor 4

[Fix automatically]

E22. Voice Input

В Chat:

🎤

Lifecycle:

recording
processing

transcribing
editable transcript
send

Обязательно:

Cancel
Pause
Resume
Retry

Не привязывать архитектуру к одному Whisper provider.

Создать:

SpeechToTextAdapter

E23. Image Generation Pipeline

Не делать:

generateImage(prompt)

как один вызов.

Создать нормальную job-модель:

prompt
reference assets
dimensions
style
brand constraints
provider
seed
metadata
status
outputs

Результаты версионируются.

E24. Video Studio Pipeline

Pipeline:

brief
↓
storyboard
↓
shots
↓
generation
↓
timeline
↓
audio
↓
render
↓
QA
↓
export

UI:

Storyboard
Timeline
Assets
Render queue
Preview
Export

E25. Music Studio

Твой список здесь хороший, но разделить реализацию.

Phase A

MIDI
structure
tempo
key
arrangement
reference metadata

Phase B

audio analysis
LUFS
spectrum
stereo
dynamics

только при наличии настоящего аудиофайла.

Phase C

stems
mix
master
render

Никаких выдуманных audio metrics.

E26. Beatport Lab

Я бы не строил систему вокруг неофициального скачивания треков.

Правильная архитектура:

metadata collector
chart history
rank history
artist/release metadata
reference links

Audio analysis только при наличии легально доступного файла/preview с подходящими правами.

E27. Resource Ledger 2.0

У тебя Resource Ledger уже не нужно создавать с нуля.

Его надо **расширять**.

Учитывать:

input tokens
output tokens
reasoning
cache
image units
video seconds
audio seconds
tool calls
browser runtime

container CPU
storage
bandwidth

E28. Budget Reservation

Это закрывает проблему eventual consistency.

Перед платным вызовом:

estimated cost = $0.18

сначала:

reserve $0.18

после результата:

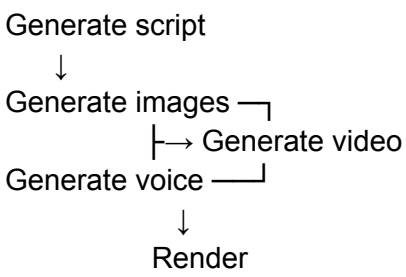
actual $0.14

release $0.04

Так параллельные агенты не смогут одновременно превысить бюджет.

E29. Mission Dependency Graph

Для мультимодальных задач:



Нужны:

dependencies
parallel groups
retry policy
timeouts
resource requirements

cost estimate

E30. Fencing Tokens

Тоже хороший пункт.

Project lock:

epoch 41

Worker A работает с:

epoch 41

проект передан Worker B:

epoch 42

После этого Worker A больше не может писать данные.

E31. Sandbox Security

Следующая серьёзная backend-серия:

network policy
domain allowlist
secret proxy
resource quota
process timeout
filesystem quota

Секрет:

OpenAI key

не должен попадать внутрь sandbox как обычная env variable, если это можно избежать.

E32. Durable Workflows

Не надо срочно переписывать всё ядро.

Сначала определить долгие операции:

video render
large coding mission
backup
migration
multistep publishing

И именно их переносить на durable workflow abstraction.

E33. Keyboard Command System

Глобальная:

⌘K / Ctrl+K

Command Palette:

Switch model
Open studio
Run mission
Stop mission
Search project
Deploy
Open history
Open connections

И:

?

открывает shortcuts.

E34. Contextual Help

Вместо сотни tooltip одновременно:

ⓘ

только рядом с сложными настройками.

Tooltips должны объяснять:

что это
что изменится
риск
стоимость

E35. Universal Status System

Один визуальный язык для:

Success
Warning
Failure
Running
Paused
Offline
Pending approval

Не должно быть так:

Studio A → зелёная точка
Studio B → текст Ready
Studio C → галочка

E36. Final Studio UX Consistency Pass

Для каждой студии проверить один стандарт:

Функция	Chat	Web	Image	Video	Music	Labs
Suggestions	✓	✓	✓	✓	✓	✓
Loading	✓	✓	✓	✓	✓	✓
Error	✓	✓	✓	✓	✓	✓
Retry	✓	✓	✓	✓	✓	✓
Cancel	✓	✓	✓	✓	✓	✓
History	✓	✓	✓	✓	✓	✓

Export	—	✓	✓	✓	✓	✓
Help	✓	✓	✓	✓	✓	✓

E37. Real E2E

Когда UX закончен:

Login
→ create project
→ open studio
→ invoke model
→ edit result
→ save
→ reload
→ continue
→ export

В браузере.

Не source-string tests.

E38. Failure Injection

Проверить:

D1 unavailable
AI provider timeout
R2 timeout
WebSocket disconnected
browser lost connection
token expired
deploy response lost
worker restarted

Платформа должна восстанавливаться.

E39. Desktop + Mobile Acceptance

Минимум:

1440px
1024px
768px
390px

Проверять:

navigation
chat
studio editor
modals
forms
suggestions
keyboard
touch

E40. Release Readiness

Только после этого:

Node24
CI
Staging
E2E
Security
Backup drill
Browser QA
Approval
GitHub
Production

Что из твоего старого промпта уже не нужно делать «с нуля»

Есть несколько пунктов, которые были верны для старых версий, но сейчас их надо **усиливать, а не создавать заново**.

Пункт из промпта	Что делать сейчас
OIDC	усиливать и завершить production cutover

Resource Ledger	расширить до Ledger 2.0
Backup	довести cloud restore drill
GitHub/Cloudflare deployment	завершать Node24/CI цепочку
Event integrity	уже усилено, дальше monitoring
Error/security architecture	продолжать runtime verification
Repository cleanup	уже сильно сделано
31 connectors	не делать 31 сразу
Legacy	хранить отдельно от runtime bundle
Production deploy	только после всех release gates

Главный приоритет следующей серии

Я бы поставил порядок именно такой:

1 Contextual Suggestions
↓
2 Universal Studio Toolbar
↓
3 History + Undo/Redo
↓
4 Loading/Error/Retry/Cancel
↓
5 Model Switcher + Auto Router
↓
6 Connector Gateway
↓
7 Web Studio Visual Editing
↓
8 Voice/Image/Video/Music pipelines
↓
9 Mission Graph + Abort + Reconciliation
↓
10 Sandbox / Billing / Reliability
↓
11 Real browser E2E
↓
12 Production

И вот это, на мой взгляд, главное изменение мышления для новой серии:

AZRAIL должен не просто иметь много функций. Он должен постоянно подсказывать пользователю лучший следующий шаг.

То есть через 6 секунд после открытия любого экрана должно быть понятно:

Где я?
Что происходит?
Что я могу сделать?
Что система рекомендует сделать дальше?

Именно **Suggestion Engine + единый toolbar + history/recovery + одинаковые states во всех студиях** я бы сделал ядром следующей серии. Это даст заметно больший продуктовый эффект, чем добавление ещё двадцати разрозненных кнопок.

Да. Это стоит добавить не как «ещё пару кнопок в Music Studio», а как отдельный **Music Intelligence subsystem** внутри следующей серии обновлений.

Я проверил актуальную информацию по Ableton, DAWproject, Camelot Wheel и библиотекам аудиоанализа. Ниже — блок, который я бы прямо вставил в наш **Series E**.

E25 — MUSIC INTELLIGENCE STUDIO 2.0

E25.1. Music Intake — загрузка одного или нескольких треков

Пользователь должен иметь возможность загрузить:

1 track
2–10 tracks
album/reference folder
stems
MIDI
DAW project

Поддержать минимум:

WAV
AIFF
FLAC
MP3
AAC
MIDI

После загрузки:

Decode
↓
Validate
↓
Waveform
↓
BPM
↓
Beat grid
↓
Key
↓
Camelot
↓
Structure
↓
Detailed analysis

Для пачки из 10 треков — отдельная очередь:

Track 1 Analyzing ▓▓▓▓▓▓▒▒
Track 2 Done
Track 3 Waiting
...
Track 10 Waiting

Не блокировать весь UI одним тяжёлым анализом.

E25.2. Track Segmenter

Вот твоя идея с «нарезать на куски» — обязательно.

Два режима.

Auto Structure

Система сама определяет:

Intro
Groove
Build
Breakdown / Яма
Pre-drop
Drop
Bridge

Second breakdown
Final drop
Outro

Manual Slicing

Пользователь пишет:

Раздели этот трек на 10 частей

или:

Режь каждые 16 bars

или:

Выдели только breakdown и drop

UI:

| INTRO | GROOVE | BUILD | BREAK | DROP | BREAK | DROP | OUTRO |

Каждый сегмент:

Play
Loop
Solo
Analyze
Add to reference
Compare
Create prompt from section

E25.3. Deep Audio Analyzer

Здесь AZRAIL должен анализировать не только BPM.

Для каждого трека:

BPM
tempo confidence
beat positions
downbeats
bar grid
time signature
global key
local key
key confidence

Camelot key
chord tendencies
tonal stability

Дальше динамика:

Integrated LUFS
Short-term LUFS
Loudness range
True peak
Crest factor
Dynamic complexity

Спектр:

sub
bass
low-mid
mid
high-mid
air

spectral centroid
spectral spread
spectral contrast
spectral rolloff
spectral flatness

Ритм:

kick density
onset density
groove stability
swing estimate
beat energy
phrase length

Стерео:

stereo width
L/R balance
correlation
mono compatibility estimate

Essentia уже предоставляет реальные алгоритмы для BPM/beat positions, EBU R128 loudness, tonal/HPCP/key, chords, onset rate и спектральных дескрипторов, поэтому это можно строить на измеряемой аналитике, а не на выдуманных числах. Essentia

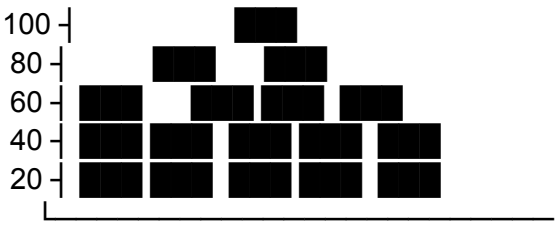
---

E25.4. Energy Curve

Для Melodic House & Techno это очень полезно.

Рисуем:

Energy



Energy рассчитывается не одной магической цифрой, а комбинацией:

loudness
spectral density
onset density
bass energy
percussive activity
arrangement density

E25.5. Camelot Wheel

Это надо сделать отдельным интерактивным виджетом.

Официальная Camelot-система использует:

A = minor
B = major

Например:

8A = A minor
8B = C major

Базовые совместимые переходы:

same:
8A → 8A

neighbor:
8A → 7A

8A → 9A

relative:
8A → 8B

Именно такие базовые движения рекомендует официальный Camelot workflow; при этом сама тональность не гарантирует хороший переход — BPM, energy, arrangement и место перехода тоже важны. Mixed In Key

UI

```
      12A
  11A       1A
10A           2A
...
```

Клик по `8A`:

Current key
A minor
Camelot 8A

Recommended:
8A
7A
9A
8B

Дополнительно:

Show compatible tracks
Sort library by Camelot
Find next harmonic reference
Build harmonic progression

E25.6. Local-Key Timeline

Глобальной тональности мало.

Нужен timeline:

00:00 8A
01:02 8A
02:18 9A
03:45 8A

Потому что breakdown, arp или bridge могут реально менять гармонический центр.

E25.7. Multi-Track Comparison

Если пользователь загрузил 10 референсов:

Reference Matrix

Track	BPM	Key	Camelot	LUFS	Energy	Breakdown	Drop
01	124	Am	8A	-8.5	82	32 bars	64
02	126	Em	9A	-7.9	88	16	64
…							

И агрегированный профиль:

Reference profile

BPM:
124–126

Most common:
125

Keys:
8A / 9A / 8B

Typical intro:
32 bars

Main breakdown:
24–32 bars

Drop:
32–64 bars

E25.8. Professional Prompt Composer

Вот здесь делаем то, что ты описал.

Не:

Сделай Melodic Techno

а автоматически строить настоящий **Production Brief**.

Например структура:

GENRE
Melodic House & Techno

TEMPO
125 BPM

KEY
A minor / Camelot 8A

STRUCTURE
32 bar intro
32 bar groove
16 bar build
32 bar breakdown
64 bar main section
...

RHYTHM
four-on-the-floor
controlled groove
...

BASS
sub-focused
...

HARMONY
minor tonal center
...

SOUND DESIGN
...

ENERGY CURVE
...

MIX TARGET
...

MASTER TARGET
...

DO NOT COPY
melodies
vocals
signature samples
exact arrangements

Кнопки:

[Build Production Prompt]
[Make More Dark]
[More Melodic]
[More Driving]
[Reduce Complexity]
[Generate Arrangement]

E25.9. Originality Guard

Очень важный архитектурный принцип.

AZRAIL учится не:

запомнить конкретную мелодию трека

а:

BPM distribution
structure
energy curve
tonality
mix balance
sound categories
arrangement statistics

То есть:

Reference tracks
↓
abstract musical features
↓
production brief
↓
new composition

а не:

Reference track
↓

copy

Это позволит использовать референсы как материал для обучения пользователя, а не как механизм клонирования чужих треков.

E25.10. Music Learning Coach

Это как раз часть:

чтобы он учился писать треки и улучшался.

После каждой версии:

Version 1
↓
Analyze
↓
Find problems
↓
Give 3 actionable improvements
↓
Version 2
↓
Compare

Например:

V2 vs V1

Bass:
+2.1 dB better balance

Breakdown:
too long by ~16 bars

Stereo:
improved

Drop energy:
+11%

Low-mid masking:
still high

Recommendation:

reduce 250–400 Hz density during drop

Так пользователь реально видит прогресс.

E25.11. Personal Music Memory Library

Да — библиотека памяти нужна обязательно.

Я бы сделал отдельный:

Music Memory

Хранить:

References
Analyses
Projects
Prompts
Presets
Arrangement templates
Mix notes
Master notes
Preferred BPM
Preferred keys
Preferred energy curves
Successful decisions
Rejected decisions

Не просто файлы.

А структурированная память:

My style profile

Typical BPM:
124–126

Preferred keys:
8A
9A
5A

Preferred structure:
long atmospheric breakdown

Preferred bass:

deep / controlled

Preferred mood:
dark / melodic / cinematic

E25.12. Memory Retrieval

В новом проекте пользователь нажимает:

Use my Music Memory

AZRAIL отвечает:

Based on your last 12 projects:

• You usually prefer 124–126 BPM
• Your stronger tracks use shorter second breakdowns
• 8A/9A harmonic centers appear most often
• You previously rejected very bright supersaws

Это уже реально персональный production assistant.

E25.13. DAW Project Import Gateway

Это надо сделать отдельным subsystem.

Не пытаться написать один гигантский parser для всех DAW.

Архитектура:

DAWImportAdapter
 ├ Ableton
 ├ DAWproject
 ├ Bitwig
 ├ Studio One
 ├ FL Studio
 ├ Logic
 ├ Cubase
 └ Generic Stems

E25.14. Ableton Live Import

Ableton официально использует:

.als = Live Set
.alp = Live Pack
.alc = Live Clip
``` :chatgpt-content-reference{index="2"}

Но Live Set может ссылаться на внешние samples, поэтому пользователь должен перед загрузкой использовать:

```text
Collect All and Save

Ableton прямо рекомендует это при переносе проектов: команда собирает audio/video и Max for Live assets в Project folder. При этом обычные plug-ins внутрь проекта не копируются и должны быть установлены отдельно. help.ableton.com

Поэтому AZRAIL должен принимать:

Ableton Project ZIP

в котором:

Project/
 ├ Song.als
 ├ Samples/
 │   └ Collected/
 ├ Ableton Project Info/
 └ Presets/

После загрузки:

Project scanner
↓
tracks
↓
audio assets
↓
MIDI
↓
tempo
↓
automation where available
↓
plugin manifest
```
```

↓
missing assets report

E25.15. DAWproject — главный универсальный формат

Вот это особенно интересно.

Существует открытый формат **DAWproject**, специально созданный для переноса проектов между DAW.

Он может хранить:

audio
notes
automation
tempo
time signature
tracks
channels
plug-in state
clip information

и использует открытое представление структуры проекта. www.bitwig.com

Поэтому архитектурно AZRAIL лучше сделать так:

Native DAW
↓
DAWproject where possible
↓
AZRAIL Project Model

Это намного правильнее, чем бесконечно reverse-engineer proprietary formats.

E25.16. Universal Fallback

Если DAW не поддерживает хороший structured import:

ZIP
├ stems/
├ midi/

```
├ master.wav
├ reference.wav
└ project.json
```

AZRAIL импортирует:

stems
MIDI
BPM
key
bar structure
markers

И создаёт внутренний проект.

E25.17. Plugin Compatibility Report

После загрузки DAW project:

Plugins detected

```
Serum          Found
Diva           Missing
FabFilter Q3   Found
Valhalla       Missing
```

И:

[Keep placeholder]
[Use audio bounce]
[Replace device]

Это особенно важно, потому что Ableton отдельно предупреждает: сторонние плагины не упаковываются обычным Collect All and Save. help.ableton.com

---

E25.18. Stem Separation

Если пользователь загрузил только master:

Vocals
Drums
Bass

Other

можно использовать pluggable stem separator.

Но я бы **не привязывал ядро AZRAIL к Demucs**: официальный Meta-репозиторий Demucs сейчас архивирован и read-only с 2025 года. Его можно оставить как один optional backend, но не как единственный production dependency. GitHub

Интерфейс:

StemSeparatorAdapter

чтобы потом менять engine без переписывания Music Studio.

E25.19. Composition Engine

После анализа:

References
↓
Feature abstraction
↓
Production Prompt
↓
Composition Plan
↓
MIDI
↓
Sound Design
↓
Stems
↓
Mix
↓
Master
↓
Final analysis

Внутри:

Chord progression
Bass MIDI
Arp
Lead
Pads
Drums

FX
Transitions
Automation

E25.20. Melodic House & Techno Mode

Сделать отдельный профиль:

Melodic House & Techno

Настройки:

BPM
Key
Camelot
Mood
Darkness
Melodic intensity
Drive
Atmosphere
Breakdown length
Drop intensity
Bass character
Percussion complexity

Но это именно **жанровый профиль**, а не копирование конкретного артиста.

E25.21. Render Quality Loop

После создания композиции:

Render
↓
Analyze
↓
Compare against target
↓
Fix
↓
Render again

До достижения:

tempo correct
key correct
no clipping
acceptable LUFS
stable low end
balanced stereo
desired energy curve

E25.22. Reference A/B Player

Очень нужная функция:

A = user's project
B = reference

Автоматически:

loudness match
start sync
loop section

Кнопки:

A
B
LUFS Match
Loop
Sync

Чтобы громкость не обманывала слух.

E25.23. Suggestion Engine внутри Music Studio

И это связывает Music Studio с нашей главной идеей Google AI Studio-style suggestions.

После анализа референса:

[Построить структуру]
[Создать production prompt]
[Найти breakdown]
[Показать Camelot]

[Сравнить с моими треками]

После анализа собственного трека:

[Найти слабые места]
[Улучшить drop]
[Очистить low-mid]
[Сделать breakdown короче]
[Подготовить master]

После DAW import:

[Проверить missing plugins]
[Analyze arrangement]
[Analyze mix]
[Find unused tracks]
[Prepare next version]

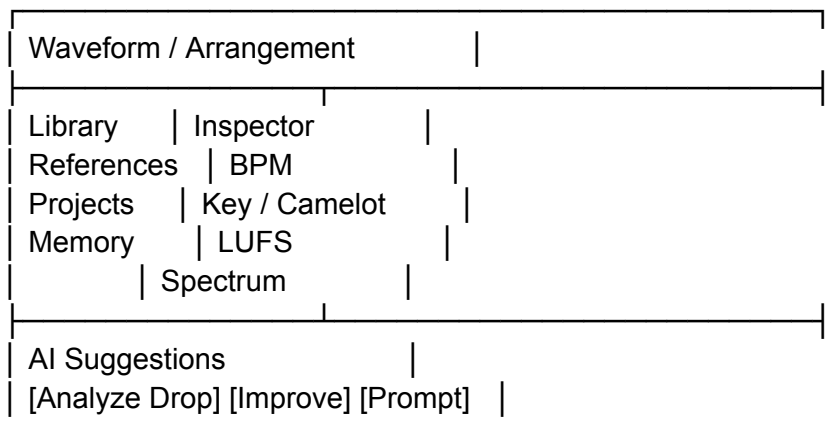
E25.24. Music Studio UI

Я вижу структуру так:

MUSIC STUDIO

Library
Analyze
Arrange
Compose
Mix
Master
Memory
Projects

Главное окно:



└──┘

Новый итоговый Music pipeline

UPLOAD
audio / stems / MIDI / DAW project
↓
INGEST
↓
SEGMENTATION
↓
DEEP ANALYSIS
↓
CAMELOT / HARMONY
↓
REFERENCE MATRIX
↓
MUSIC MEMORY
↓
PRODUCTION PROMPT
↓
COMPOSITION PLAN
↓
MIDI / ARRANGEMENT
↓
SYNTHESIS
↓
STEMS
↓
MIX
↓
MASTER
↓
FINAL ANALYSIS
↓
LEARNING COACH
↓
MEMORY UPDATE

Что я бы поменял в нашем общем Series E

Теперь блок приоритетов становится таким:

1. Contextual Suggestions
2. Universal Studio Toolbar

3. History / Undo / Recovery
4. Model Router
5. Connector Gateway
6. Web Studio
7. MUSIC INTELLIGENCE STUDIO 2.0
8. Voice / Image / Video
9. Mission Graph
10. Resource Ledger
11. Sandbox Security
12. Real E2E
13. Production

И **Music Intelligence Studio** я бы уже не считал обычной «студией». Это отдельная полноценная система:

Analyzer
+
Music Theory Engine
+
Camelot
+
Reference Library
+
Memory
+
Prompt Composer
+
DAW Project Importer
+
Composition Coach
+
Renderer

Это уже очень сильное направление для AZRAIL.

Да. Я бы оформил это как отдельную большую серию **Series F — Studio Intelligence Expansion**. Здесь уже не просто добавляем кнопки, а доводим Music, Image, Video, Metatron и модульную систему до уровня профессиональной рабочей среды.

F01–F20 — Music Intelligence Studio

1. **Multi-track intake** — загрузка 1–20 треков, stems, MIDI и DAW-проектов.
2. **Smart segmentation** — автоматическое определение intro, build, breakdown, drop, outro.
3. **Manual slicing** — нарезка по bars, времени или пользовательским маркерам.

4. **Deep key analysis** — глобальная и локальная тональность по секциям.
5. **Camelot Wheel** — интерактивный круг с harmonic compatibility.
6. **Chord map** — предполагаемая гармоническая карта трека.
7. **Energy curve** — визуальная карта энергии по времени.
8. **LUFS / dynamics panel** — Integrated/Short-term LUFS, true peak, crest factor.
9. **Spectrum analyzer** — sub, bass, low-mid, mid, highs, air.
10. **Stereo analysis** — width, phase correlation, mono compatibility.
11. **Groove analyzer** — kick density, swing, onset density, rhythmic stability.
12. **Reference matrix** — сравнение 2–20 треков в одной таблице.
13. **A/B loudness-matched player** — сравнение собственного трека с reference без обмана громкостью.
14. **Professional Production Prompt Builder** — подробный production brief из измерений треков.
15. **Music Learning Coach** — конкретно показывает, что улучшилось между V1/V2/V3.
16. **Music Memory Library** — хранит проекты, анализы, preferences, любимые tonal/arrangement решения.
17. **DAW Project Import** — Ableton, DAWproject и fallback через stems/MIDI.
18. **Plugin Compatibility Scanner** — missing plug-ins, bounced tracks, replacement suggestions.
19. **Composition Engine** — chord/MIDI/bass/arp/drums/FX/arrangement plan.
20. **Render → Analyze → Improve Loop** — финальный трек снова анализируется и автоматически формирует следующую итерацию.

Дополнительно я бы ввёл **Style DNA**, но не в смысле копирования артиста. Это агрегированный профиль:

tempo range
tonality
energy
arrangement density
bass profile
drum complexity
breakdown length
spectral balance

Так AZRAIL учится именно принципам продакшна.

F21–F40 — Image Studio 2.0

1. **Reference Board** — несколько изображений в одном проекте.
2. **Smart Prompt Suggestions** — подсказки исходя из текущего изображения и выбранного объекта.
3. **Model Quick Switcher** — быстрый выбор image model.
4. **Auto Model Router** — система сама выбирает модель под edit/generate/vector/upscale.

5. **Layers Panel** — изображения, текст, masks, effects, background.
6. **Mask Editor** — brush, erase, feather, invert.
7. **Inpainting** — изменение выделенной области.
8. **Outpainting** — расширение изображения за границы canvas.
9. **Object Remove** — удаление объекта с реконструкцией фона.
10. **Object Replace** — замена выбранного объекта.
11. **Background Replace** — автоматический segmentation + новый background.
12. **Variants Grid** — 4/8/16 вариантов с сохранением genealogy.
13. **Image Version History** — V1 → V2 → V3 с визуальным diff.
14. **Style Lock** — сохранять визуальный язык между генерациями.
15. **Character/Product Consistency** — reference identity внутри проекта.
16. **Brand Constraints** — logo, colors, typography, visual restrictions.
17. **Structural UI Mode** — таблицы/UI/диаграммы создаются структурно, а не рисуются растром.
18. **Upscale + Restoration Pipeline** — отдельный post-processing stage.
19. **Image QA Agent** — проверка артефактов, текста, cropping, contrast, resolution.
20. **Export Center** — PNG/JPEG/WebP/SVG where applicable + social/device presets.

Очень сильная дополнительная функция — **Region Suggestions**.

Пользователь выделяет объект, а снизу появляются:

[Remove]
[Replace]
[Change color]
[Make darker]
[Match background]
[Create variants]

Это сильно улучшает 6-секундное правило.

F41–F60 — Video Studio 2.0

1. **Project Storyboard**.
2. **Shot List**.
3. **Timeline Editor**.
4. **Scene Cards**.
5. **Asset Library**.
6. **Reference Video Board**.
7. **Prompt Suggestions per shot**.
8. **Shot-level model selection**.
9. **Character/visual consistency controls**.
10. **Camera movement controls** — pan, tilt, dolly, zoom, orbit.
11. **Keyframe controls**.
12. **Transition editor**.
13. **Voice-over track**.

14. **Music / SFX tracks**.
15. **Automatic subtitles/captions**.
16. **Color matching / grading stage**.
17. **Render Queue**.
18. **Proxy Preview** — быстрые low-resolution previews до финального render.
19. **Video QA Agent** — flicker, continuity, frame errors, caption overlap, audio clipping.
20. **Export Profiles** — 16:9, 9:16, 1:1, 4K, web/social presets.

Дополнительно нужен **Continuity Memory**:

character
clothing
lighting
location
camera
props
color palette

между сценами.

Иначе AI-video выглядит как набор несвязанных генераций.

F61–F80 — Metatron Lab 2.0

Metatron я бы превратил в настоящий **Agent Operations Center**.

1. **Live Mission Graph** — визуальный DAG всех задач.
2. **Agent Cards** — кто что делает прямо сейчас.
3. **Start / Pause / Resume / Cancel / Terminate**.
4. **Real AbortSignal** до provider request.
5. **Live Logs** через SSE/WebSocket.
6. **Trace Explorer** — mission → agent → tool → provider → response.
7. **Real-time token/cost/resource display**.
8. **Checkpoint System**.
9. **Resume from Checkpoint**.
10. **Mission Replay**.
11. **Step Retry** вместо полного перезапуска.
12. **uncertain_effect state**.
13. **Effect Reconciliation Engine**.
14. **Fencing Tokens**.
15. **Agent Health Monitoring**.
16. **Automatic Agent Replacement** при падении.
17. **Human Approval Nodes** внутри mission graph.
18. **Fault Injection Lab** — timeout, provider down, D1 failure, network loss.
19. **Memory Inspector** — какая память была использована агентом.
20. **Decision Explanation Panel** — почему выбрана модель/tool/route.

Очень важный экран:

MISSION

Planner DONE
Coder RUNNING
QA WAITING
Deploy Agent BLOCKED / approval required

Cost $0.17
Elapsed 02:43
Risk LOW

[Pause]
[Stop]
[Open graph]
[Open logs]

То есть Metatron должен быть не просто красивым окном с логами, а **control plane всей агентной системы**.

F81–F100 — Universal Module System 2.0

Все модули AZRAIL должны работать по одному стандарту.

1. **Drag handle / Grip**.
2. **Collapse / Expand**.
3. **Duplicate**.
4. **Delete**.
5. **Pin**.
6. **Lock**.
7. **Rename**.
8. **Move between workspaces**.
9. **Module Settings**.
10. **Reset to defaults**.
11. **Undo / Redo support**.
12. **Loading state**.
13. **Error state**.
14. **Retry state**.
15. **Live status badge**.
16. **Permission-aware actions**.
17. **Module-specific keyboard shortcuts**.
18. **Module History**.

19. **Resource Usage indicator**.
20. **Extensible action API** для plugins.

Я бы ввёл единый контракт:

```
ModuleDefinition {
  id
  type
  title
  status
  capabilities
  actions
  permissions
  resourceUsage
  history
}
```

И каждый модуль использует один:

ModuleFrame

а не рисует свой собственный header, spinner, delete button и error message.

Ещё что реально стоит добавить

Следующая серия будет намного сильнее, если поверх этих 100 улучшений добавить несколько сквозных систем.

Smart Suggestion Engine

Он должен работать **везде**:

Chat
Music
Image
Video
Web
Metatron
Labs
Modules

Пример Music:

[Найти тональность]
[Показать Camelot]
[Сравнить breakdown]

[Подготовить production prompt]

Image:

[Убрать фон]
[Сделать варианты]
[Исправить текст]
[Поднять качество]

Video:

[Создать следующий shot]
[Исправить continuity]
[Добавить voice]
[Render preview]

Metatron:

[Повторить failed step]
[Открыть trace]
[Продолжить с checkpoint]
[Снизить стоимость]

Это должно стать **центральным UX-принципом AZRAIL**.

Universal Command Palette

Ctrl+K / Cmd+K

и затем:

Open Music Studio
Analyze track
Switch model
Create image
Run mission
Open history
Open Metatron
Deploy
Search files

Global Project Memory

Не отдельная память Music, Image и Video, которые ничего не знают друг о друге.

Нужна иерархия:

AZRAIL Memory
├ User Memory
├ Workspace Memory
├ Project Memory
├ Music Memory
├ Image Memory
├ Video Memory
├ Agent Memory
└ Temporary Session Memory

С понятной политикой:

what may be stored
why it was stored
scope
TTL
source
confidence

Asset Intelligence Library

Единая библиотека:

Audio
Images
Videos
Documents
Prompts
MIDI
Stems
DAW projects
Models
Presets

Каждый asset автоматически получает:

metadata
tags
embedding
project links
usage history
versions
analysis

Compare Mode

Очень сильная универсальная функция.

В любой студии:

A | B

Music:

Track V1 | Track V2

Image:

Original | Edit

Video:

Render V2 | Render V3

Web:

Before | After

Prompt:

Prompt A | Prompt B

Professional Review Agent

После завершения любой работы:

REVIEW

и система проверяет не просто «готово/не готово», а выдаёт:

Critical
Major
Minor
Improvement

и:

[Fix critical]
[Fix all safe issues]
[Review manually]

Как я бы оформил следующую серию полностью

SERIES F

F01–F20
Music Intelligence Studio

F21–F40
Image Studio 2.0

F41–F60
Video Studio 2.0

F61–F80
Metatron Lab 2.0

F81–F100
Universal Module System

А затем:

SERIES G

G01 Suggestion Engine
G02 Global Memory
G03 Asset Library
G04 Command Palette
G05 History / Undo
G06 Connector Gateway
G07 Resource Ledger 2.0
G08 Mission Graph
G09 Browser QA
G10 Real E2E

Так у нас получится не просто «добавить 100 функций», а единая архитектура: **каждая студия профессиональная сама по себе, а Suggestions, Memory, History, Models, Assets и Metatron объединяют их в одну операционную систему.**